



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

Organización de Lenguajes y Compiladores 1

Sección C

Escuela de Ingeniería de Ciencias Y Sistemas

Johnny Whillman Aldana Osorio



UNIDAD #10:

Manejo de entornos

**Fundamentos para construir intérpretes
con alcance y memoria**

Unidad 10



Agenda

- ¿Qué es un entorno dentro de un intérprete?
- Diferencia entre tabla de símbolos y entorno de ejecución.
- Alcance de variables: global, local y bloque.
- Implementación de entornos con clases en TypeScript.
- Ejemplo práctico y actividad en clase.



COMPETENCIA

Competencia que desarrollaremos

- Diseñar e implementar estructuras de soporte para un intérprete, permitiendo almacenar, consultar y actualizar símbolos durante la ejecución de un programa.
- Relacionar el análisis semántico con la ejecución de instrucciones mediante el uso de entornos encadenados.

MARCO FORMATIVO

Valor de la semana



- Responsabilidad: al implementar un intérprete, cada variable, función o símbolo debe administrarse correctamente para evitar resultados incorrectos.
- El manejo ordenado de entornos ayuda a detectar errores, mantener claridad en el código y respetar las reglas del lenguaje.

CONCEPTO BASE

¿Qué es un entorno?



- Un entorno es una estructura que guarda asociaciones entre nombres y valores.
- Permite saber qué valor tiene una variable durante la ejecución.
- Puede tener un entorno padre para representar alcances anidados.
- Es la memoria lógica que utiliza el intérprete para resolver identificadores.

DIFERENCIA IMPORTANTE

Tabla de símbolos vs. entorno



Tabla de símbolos

- Se usa principalmente durante el análisis semántico.
- Guarda información como nombre, tipo, ámbito y línea.
- Ayuda a validar declaraciones y usos correctos.

Entorno de ejecución

- Se usa al interpretar o ejecutar instrucciones.
- Guarda valores actuales de variables.
- Puede cambiar durante la ejecución del programa.

USO EN INTÉRPRETES

¿Por qué es importante?



- Evita que el intérprete pierda el valor de las variables.
- Permite ejecutar asignaciones, operaciones y llamadas a funciones.
- Facilita el manejo de errores como variables no definidas.
- Permite implementar alcance local y global de forma ordenada.



SCOPE

Alcance de variables

- Global: disponible en todo el programa.
- Local: disponible dentro de una función o bloque específico.
- Bloque: disponible solamente dentro de estructuras como if, while o for.
- Shadowing: ocurre cuando una variable local tiene el mismo nombre que una variable externa.

MODELO MENTAL

Entornos anidados

Global

- $\pi = 3.14$
- saludo = "hola"

Función calcular()

- $x = 10$
- $y = 20$

Bloque if

- temp = 30



Regla: buscar primero en el entorno actual y luego en el padre

LOOKUP

Reglas de búsqueda



- Primero se busca el identificador en el entorno actual.
- Si no existe, se busca en el entorno padre.
- El proceso continúa hasta llegar al entorno global.
- Si no se encuentra, se reporta un error semántico o de ejecución.

IMPLEMENTACIÓN

Estructura base en TypeScript



Un entorno puede implementarse como una clase con un mapa interno y una referencia opcional al entorno padre.

```
class Entorno {  
    private valores: Map<string, any> = new Map();  
  
    constructor(private anterior: Entorno | null = null) {}  
  
    declarar(nombre: string, valor: any): void {  
        this.valores.set(nombre, valor);  
    }  
}
```

Idea clave El constructor permite crear entornos hijos conectados al entorno anterior.

LOOKUP

Obtener una variable



La búsqueda sube por la cadena de entornos hasta encontrar el identificador.

```
obtener(nombre: string): any {  
  if (this.valores.has(nombre)) {  
    return this.valores.get(nombre);  
  }  
  
  if (this.anterior !== null) {  
    return this.anterior.obtener(nombre);  
  }  
  
  throw new Error(`Variable ${nombre} no definida`);  
}
```

Idea clave Este método representa la regla de búsqueda del intérprete.



ACTUALIZACIÓN

Asignar una variable

La asignación debe modificar la variable en el entorno donde fue declarada.

```
asignar(nombre: string, valor: any): void {  
  if (this.valores.has(nombre)) {  
    this.valores.set(nombre, valor);  
    return;  
  }  
  
  if (this.anterior !== null) {  
    this.anterior.asignar(nombre, valor);  
    return;  
  }  
  
  throw new Error(`Variable ${nombre} no definida`);  
}
```

Idea clave: Declarar crea una variable; asignar actualiza una variable existente.

EJEMPLO

Uso del entorno global y local



Este ejemplo muestra cómo un entorno local puede consultar valores del entorno global.

```
const global = new Entorno();
global.declarar("x", 10);
global.declarar("mensaje", "Hola");

const local = new Entorno(global);
local.declarar("y", 20);

console.log(local.obtener("y")); // 20
console.log(local.obtener("x")); // 10
```

Idea clave El entorno local no tiene x, pero puede encontrarla en el entorno global.

ALCANCE

Shadowing



Una variable local puede ocultar temporalmente a una variable externa con el mismo nombre.

```
const global = new Entorno();  
global.declarar("x", 10);  
  
const bloque = new Entorno(global);  
bloque.declarar("x", 99);  
  
console.log(bloque.obtener("x")); // 99  
console.log(global.obtener("x")); // 10
```

Idea clave El intérprete usa primero el valor del entorno más cercano.



FUNCIONES

Entornos en funciones

- Cada llamada a una función puede crear un nuevo entorno local.
- Los parámetros se declaran dentro del entorno de la función.
- Las variables internas no deberían contaminar el entorno global.
- Al finalizar la función, su entorno local deja de utilizarse.

CUIDADO

Errores comunes



- Usar una variable que nunca fue declarada.
- Declarar dos veces la misma variable en el mismo entorno si el lenguaje no lo permite.
- Actualizar una variable en el entorno equivocado.
- No crear un entorno nuevo para bloques o funciones cuando sí corresponde.

RECOMENDACIONES

Buenas prácticas



- Separar la lógica del entorno de la lógica de las instrucciones.
- Usar nombres claros para métodos como declarar, obtener y asignar.
- Reportar errores con línea y columna cuando sea posible.
- Probar casos con entorno global, local, shadowing y variables no definidas.



Ejemplo práctico

Mini intérprete con variables y entornos

- Crear un entorno global.
- Declarar variables y evaluar expresiones.
- Crear un entorno hijo para simular un bloque.
- Comprobar shadowing y errores de variable no definida.

EN CLASE

Mini reto guiado



Completa las instrucciones para observar cómo cambia el valor de las variables.

```
const global = new Entorno();
global.declarar("contador", 0);

global.asignar("contador", global.obtener("contador") + 1);

const ciclo = new Entorno(global);
ciclo.declarar("i", 1);

console.log(global.obtener("contador"));
console.log(ciclo.obtener("i"));
```

Idea clave El objetivo es identificar qué variable pertenece a cada entorno.



Practiquemos lo aprendido

- Implementa una clase Entorno con `declarar()`, `asignar()` y `obtener()`.
- Prueba un entorno global y un entorno local con el mismo nombre de variable.
- Muestra en consola el valor que se obtiene antes y después de modificar una variable.
- Entrega una captura del resultado y el archivo TypeScript.

RESUMEN

Conceptos clave aprendidos



- Un entorno guarda nombres y valores durante la ejecución.
- Los entornos pueden encadenarse para representar alcance.
- La búsqueda inicia en el entorno actual y continúa hacia el padre.
- La tabla de símbolos valida; el entorno ejecuta y conserva valores.
- El shadowing permite que una variable local oculte una variable externa.

BIBLIOGRAFÍA

Referencias



- Aho, A., Lam, M., Sethi, R. & Ullman, J. Compilers: Principles, Techniques & Tools.
- Gamma, E., Helm, R., Johnson, R. & Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software.
- Loudon, K. Lenguajes de programación: principios y práctica.
- Material de apoyo del curso de Organización de Lenguajes y Compiladores 1.



DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

Recuerda consultar el foro semanal ante cualquier duda de la sesión.